



Events as Infrastructure — Why Real-Time Operations Need More Than Webhooks

Mimir Labs Technical Publication

Document: ML-WP-015 **Version:** 1.0
Author: Christopher Gaither **Date:** April 2026
Classification: Public

1 Events as Infrastructure — Why Real-Time Operations Need More Than Webhooks

Mimir Labs Publication Document Version: 1.0 **Date:** April 2026 **Classification:** Public

1.1 Summary

Enterprise software has spent twenty years treating real-time event flow as a feature rather than as infrastructure. The result is an industry that ships webhooks, sells event meshes as separate products, and routes integration concerns through a tax layer of iPaaS vendors that exist because the underlying systems cannot speak directly to each other.

Yggdrasil ERP takes a different position. Real-time event distribution is not a feature of the platform; it is part of the platform. The B2B Event Hub is built into the server, governed by the same authorization envelope that protects the rest of the API surface, persisted to the same canonical database, and federated for multi-node deployments through pluggable relays. There is no separate SKU. There is no third-party middleware in the path. There is no per-event charge.

This paper explains why that distinction matters, what it changes for enterprises that buy Yggdrasil, and how it compares to the integration approaches the rest of the industry sells.



1.2 The Problem with How the Industry Ships Events

Most enterprise systems were designed in an era when real-time visibility into business state was a luxury. Operational data lived in the transactional system, analytical data lived in a warehouse, and the gap between them was closed by a nightly extract-transform-load pipeline. The CEO asked how the business was doing, and the answer was yesterday's data with a PowerPoint timestamp on it.

Real-time integration was bolted on later, as a response to customer demand for tighter system coupling. The bolt-on took three forms, each of which failed to address the underlying problem.

Webhooks were added as an afterthought. A webhook fires when something happens; it tells the receiver that something changed but rarely what changed. The receiver fetches the affected resource, fetches related resources, computes a diff, and acts on the result. Pulling a thousand orders takes a thousand HTTP round trips. The vendor charges for the API calls.

Event meshes were sold as separate products. Major ERP vendors recognized that integration through webhooks did not scale, and responded by introducing event mesh products as additional SKUs. SAP Event Mesh, Salesforce Platform Events, NetSuite SuiteAnalytics — each carries its own licensing, its own implementation cost, and its own integration with the underlying system.

iPaaS became an industry. Mulesoft, Boomi, Workato, Tray, and Zapier exist because enterprise systems cannot speak directly to each other. Customers pay these vendors to build and maintain the connectors that the system vendors should have shipped. The cost is recurring, the integrations are brittle under upgrades, and the customer pays for the same integration to be rebuilt every time either side of the connection ships a new version.

The economic logic of this arrangement favors every participant in the supply chain except the customer. The system vendor does not have to invest in real-time architecture because event meshes are sold separately. The event mesh vendor does not have to invest in connectors because iPaaS handles them. The iPaaS vendor does not have to invest in semantic alignment because that is the customer's responsibility. The customer pays four bills for what should be one capability.

1.3 What an Honest Event Surface Looks Like

The right design is the one that the industry has consistently chosen not to ship.

Events are part of the platform. When the platform ships, the event surface ships with it. There is no separate license, no separate hosting, and no separate authorization model. The same identity that authenticates HTTP requests authenticates event subscriptions.

Events carry structured payloads, not occurrence pings. When something changes, the event delivers the structured representation of what changed. Subscribers do not need to refetch. They do not need to compute a diff. They receive the change as a first-class message.



Events are persisted in the same database as the rest of the system. There is no separate event store to back up, no second source of truth to reconcile, and no risk that events and database state diverge. The persisted event record is a row in the same PostgreSQL database that holds every other entity. It participates in the same backups, the same replication, and the same operational tooling.

Events are durable. Subscribers that disconnect, fail, or come online late receive the events they missed. The platform retries delivery within a bounded budget and keeps a permanent record of every persisted event. Audit replication and replay are first-class operations, not premium features.

Events are tenant-scoped by structure, not by configuration. Every authenticated subscription is bound to its tenant context at the protocol layer. Cross-tenant publication exists, but it is an explicit operation that requires an established relationship between the tenants. There is no path by which a misconfigured filter or an application bug leaks one tenant's events to another.

This is the design Yggdrasil ships. It is not novel; it is the obvious design. The reason the industry does not ship it is not technical.

1.4 What This Changes for the Customer

The architectural distinction matters at procurement time and at operational time.

1.4.1 At Procurement

The total cost of ownership for an event-driven workflow on a mainstream ERP includes the platform license, the event mesh license, an iPaaS subscription, and the implementation cost of building each connector. None of these are optional for a customer that needs operational visibility across systems. All of them carry recurring costs.

The total cost of ownership for the same workflow on Yggdrasil includes the platform license. The event surface, the persistence, the federation, and the authorization are part of the platform. The customer pays for one product and gets the capability the others sell separately.

This compounds over time. A customer who needs to add a new integration on a mainstream ERP returns to the iPaaS vendor for a new connector, returns to the event mesh vendor for additional throughput, and returns to the system integrator to wire it together. The same customer on Yggdrasil writes a WebSocket subscriber that authenticates with the existing tenant credentials and consumes structured JSON events through the existing protocol.

1.4.2 At Operations

Real-time visibility is not the headline feature of the hub; it is the byproduct of building event flow into the architecture rather than around it.



When a sales operator confirms an order, every operator viewing the same tenant's pipeline observes the transition immediately. When a quality inspector opens a nonconformance report, the manufacturing supervisor sees the alert without refreshing a dashboard. When a purchase order is received, the warehouse and the finance modules update synchronously. There is no nightly batch, no polling layer, and no perceptual lag between operational reality and the system's representation of it.

For B2B workflows the value compounds. A supplier publishes a shipment notification to a buyer tenant; the buyer's receiving floor sees the inbound shipment in real time. The buyer publishes a payment confirmation back to the supplier; the supplier's finance team sees the payment in the same instant the buyer's books reflect it. The hub treats inter-tenant communication as a first-class operation, with the same persistence, retry, and audit guarantees that intra-tenant events receive.

1.4.3 At Audit

The persisted event stream is an independent record of every governed mutation in the system. It is append-only from the publisher's perspective, persisted in the canonical database, and replayable through a query rather than through a backup restoration.

For regulated industries this is not a convenience. It is the difference between maintaining an audit trail by configuration and having an audit trail by architecture. A configuration-based audit trail can be turned off, can drift out of sync with the operational state, and can omit changes that bypass the audited path. An architectural audit trail captures every change because there is no other path through which changes occur.

1.5 How This Compares

The honest comparison is segment-specific.

Against mainstream ERP webhooks. The hub is functionally superior. Webhooks are a notification primitive; the hub is an event distribution system with persistence, retry, and federated delivery. The cost difference favors the hub. The operational simplicity favors the hub.

Against vendor event mesh products. The hub provides comparable functionality with lower architectural overhead. Event meshes are sold as add-ons to systems that did not originally include real-time event flow; the hub is a built-in component of a system that did. For organizations that already license an event mesh, the hub does not displace the broader mesh — it makes Yggdrasil's participation in the mesh straightforward through the Redpanda relay.

Against iPaaS. The hub does not replace iPaaS for organizations whose primary integration challenge is bridging multiple non-Yggdrasil systems. iPaaS earns its place when the customer is integrating six systems with eight idiosyncrasies and needs hosted middleware to manage the cross product. The hub removes the need for iPaaS as an intermediary between Yggdrasil and any subscriber that can speak WebSocket. For organizations whose integration scope is "Yggdrasil plus a



few subscribers,” the hub eliminates the iPaaS line item entirely. For organizations with broader integration estate, Bifrost — Yggdrasil’s first-party live-data-bridge product — uses the hub as its event substrate and handles the connector and routing logic for external system pairings.

The hub is not designed to be the universal integration solution for an enterprise. It is designed to make Yggdrasil’s operational state continuously observable through one protocol, governed by one authorization envelope, persisted in one database. Everything downstream of that observability — connector logic, transformation rules, multi-system orchestration — is the responsibility of higher-level tools that consume the hub.

1.6 What It Says About How Yggdrasil Is Built

The hub is one architectural choice; it is also a signal about how the platform is built generally.

Yggdrasil treats integration as a first-class concern rather than as a partner-channel opportunity. The platform ships the migration tool (Ragnarok), the integration tool (Bifrost), and the event surface (the hub) as part of the product, not as separately-licensed modules. The architectural premise is that the work an enterprise does at the boundaries of its ERP — getting data in, getting data out, observing operational state — is part of running an ERP, not a category of vendor opportunity.

The mainstream’s response to integration is to charge for it: webhook calls priced per record, event mesh licensed as a separate SKU, iPaaS subscribed by API call volume, system integrator engaged by the hour. Each of these arrangements compounds the cost of operating an integrated business.

The hub is a small example of a larger architectural commitment. The cost of running Yggdrasil reflects the cost of running the platform; the cost of integrating with it does not become its own bill.

1.7 Closing

Real-time operational visibility is not a feature. It is infrastructure. Treating it as a feature produces the architectural fragmentation that the rest of the industry has spent two decades selling, layered, to its customers. Treating it as infrastructure produces a platform whose operational state is continuously visible, whose integration cost is bounded, and whose audit posture is a structural property rather than a configurable behavior.

The B2B Event Hub is the component that makes that posture concrete. It is the substrate through which Yggdrasil becomes observable in real time — to its own clients, to its integration partners, to other tenants in B2B workflows, and to downstream analytical infrastructure. It exists as part of the platform because the platform was designed around the assumption that real-time operations are not optional.



Customers who buy Yggdrasil are not buying a license to a transactional database with optional real-time bolt-ons. They are buying a system whose operational state is continuously available to anyone authorized to see it, through one protocol, with one authorization envelope, and at no separate cost.

That is what an event surface looks like when it is treated as infrastructure rather than as a feature.

Copyright 2026 Mimir Labs. All rights reserved.

Mimir Labs LLC | Harrisburg, PA

© 2024–2026 Mimir Labs LLC. All rights reserved.

This document contains proprietary information belonging to Mimir Labs LLC. No part of this publication may be reproduced, distributed, or transmitted in any form without the prior written permission of Mimir Labs LLC. The information herein is provided “as is” without warranty of any kind. Product names and trademarks referenced in this document are the property of their respective owners.